计算机系统原理实验报告

第一次实验
基于 RISC-V 多平台的多线程与向量程序设计、优化及性能分析

成员： 江李阳 202400460042

王健一 202400460037

邱泽辉 202400460046

史淇 202333180074

专业： 密码科学与技术

分工： 江李阳：编写代码，修改润色实验报告
王健一：编写代码，撰写实验报告
邱泽辉：查找文献，撰写实验报告
史淇：实验报告整合

日期： 2026 年 4 月 7 日

Contents

1 实验一 **2**
1.1 实验环境 2
1.2 实验目的 2
1.3 实验原理 2
1.4 实验设计 3
1.5 实验过程 3
1.6 实验结果 4
1.7 实验结果分析 6
1.8 实验结论 7

2 实验二 **8**
2.1 实验环境 8
2.2 实验目的 8
2.3 实验原理 9
2.4 实验设计 9
2.5 实验过程 10
2.6 实验结果 13
2.7 实验结果分析 14
2.8 实验结论 15

A 实验相关代码 **16**
A.1 任务二基线版本代码 16
A.2 任务二优化一代码 17
A.3 任务二优化二代码 19
A.4 任务二优化三代码 21
A.5 任务二优化四代码 23
A.6 任务二编译器优化尝试代码 25

1 实验一

1.1 实验环境

- 硬件环境:
  - 处理器:13th Gen Intel(R) Core(TM) i5-13420H(12CPUs), 2.1GHz
  - 内存:16384MB RAM
- 软件环境:
  - 操作系统:Windows11
  - VMware Workstation Pro 17:Ubuntu 22.04 LTS 系统,7.8GB 内存,4 处理器,300GB 硬盘
  - Arch Linux x86_64(内核:Linux 6.6.87.2-microsoft-standard-WSL2)
    - L1d cache:288KiB(6 instances)
    - L1i cache:192 KiB(6 instances)
    - L2 cache:7.5 MiB(6 instances)
    - L3 cache:11MiB(1 instances)

1.2 实验目的

- 学习矩阵乘法程序优化: 在考虑数据大小, 线程数, 硬件配置, 程序算法等的前提下考虑矩阵乘法程序的优化.
- 理解 Cache 机制: 通过改变访存方式, 验证时间局部性与空间局部性对程序执行效率的影响. 进而最终理解 Cache 对程序执行效率的巨大影响.
- 探索并行计算机制: 利用多线程与指令级并行 (SIMD/循环展开), 分析硬件资源并行优化带来的加速比.
- 掌握跨平台工程能力: 学习交叉编译和 QEMU 模拟器的使用, 学习掌握 RISC-V 工具链的基本使用方法. 学习 RISC-V Linux 内核编译并通过 QEMU 运行.

1.3 实验原理

矩阵乘法 $C_{m\times n} = A_{m\times k} \times B_{k\times n}$, 要求第一个矩阵的列数等于第一个矩阵的行数, 一般的实现方法为 3 层循环实现, 最内层最内层循环分别遍历 A 的行和 B 的列, 其算法时间复杂度为 $O(N^3)$. 在现代计算机体系结构下, 制约其性能的核心因素通常不是计算速度, 而是存储层级:

- 空间局部性: 当访问矩阵 B 时将固定列而遍历行, 对于按行优先存储的编程语言, 这将跳跃访问内存地址, 空间局部性差.
- 当矩阵的规模较大而超过 L/L2Cache 的大小时, 内存 MISS 率将大大提高, 程序执行效率低下. 通过矩阵分块, 使分块后的矩阵能完整装入 L1/L2Cache, 将减少对主存的访问.
- 多核并行: 通过将维度任务划分, 平衡 CPU 的核心负载, 避免因负载不均导致的程序整体效率低.

1.4 实验设计

本实验一共考虑并设计了 10 个版本的矩阵乘法代码, 每个版本都在不同矩阵大小规模和线程数的条件下运行并测量结果, 其中实验设计如下:

Table 1: 矩阵乘法优化版本迭代说明

版本分类	核心技术	优化逻辑说明
基础版 [版本 0]	Naive	最原始的 i-j-k 嵌套, 访存步长较大,Cache Miss 极高
访存优化 [版本 1]	IKJ 重排	改变循环顺序, 使内层循环对 B 和 C 均为连续内存访问
访存优化 [版本 2]	矩阵转置	预先转置矩阵 B, 将纵向访问变为横向连续访问
访存优化 [版本 3]	分块 (Blocking)	利用分块技术强行提高时间局部性, 适配 Cache 容量
并行优化 [版本 4]	多线程 +IKJ	将矩阵行拆分给多个线程, 并行执行最高效的重排算法
并行优化 [版本 5]	多线程 + 分块	结合线程并行与 Cache 优化, 处理超大规模矩阵
并行优化 [版本 8]	全优化 (并行 + 转置)	融合转置带来的访存优势与多线程的计算优势
指令优化 [版本 6/9]	循环展开/向量化	减少循环跳转开销, 利用编译器自动向量化压榨流水线
指令优化 [版本 10]	手动 AVX2(仅 WSL)	手动调用 Intel 指令集, 单指令多数据并行处理

1.5 实验过程

编写对应各个版本矩阵乘代码, 考虑到模拟环境性能较差, 运行速度较慢, 所以首先在宿主机 WSL2(Arch Linux) 上运行测试 (考虑到不同版本的代码在不同平台测试结果相差较大, 但在仙女沟通平台应呈现出一致的趋势). 实验过程截图如图:

```
[root@LYJ code]# ./matmul_all
==================== Matrix Multiplication Benchmark ====================
CPU: 6 Cores | High Precision Timer (Nanosecond)
Sizes: 32/64/128/256/512/1024/2048
=========================================================================
Ver          Size    Th      Block       Time(s)      GFLOPS       Note
v0           32      1       -           0.000021     3.14         Baseline
v1           32      1       -           0.000009     7.11         IKJ
v2           32      1       -           0.000024     2.72         Transpose
v3           32      1       32          0.000009     6.93         Block
v3           32      1       64          0.000009     7.10         Block
v3           32      1       128         0.000009     7.17         Block
v3           32      1       256         0.000009     7.09         Block
v4           32      1       -           0.009365     0.01         MT-IKJ
v4           32      2       -           0.000284     0.23         MT-IKJ
v4           32      4       -           0.000395     0.17         MT-IKJ
v4           32      6       -           0.000510     0.13         MT-IKJ
v4           32      8       -           0.000439     0.15         MT-IKJ
v4           32      12      -           0.000668     0.10         MT-IKJ
v5           32      1       32          0.000118     0.56         MT-Block
v5           32      1       64          0.000118     0.55         MT-Block
v5           32      1       128         0.000119     0.55         MT-Block
v5           32      1       256         0.000150     0.44         MT-Block
v5           32      2       32          0.000144     0.45         MT-Block
v5           32      2       64          0.000120     0.54         MT-Block
v5           32      2       128         0.000128     0.51         MT-Block
```

Table 2: 实验一运行过程截图 (宿主机)

实验代码在宿主机上成功运行得到准确数据后, 迁移至 VMware 的 ubuntu 虚拟机上, 在 QEMU 环境上测试运行。由于软件模拟导致性能远小于在宿主机上运行, 因此适当减小了实验数据, 并删去了 X86 特有的 AVX2 指令集, 实验过程截图如图:

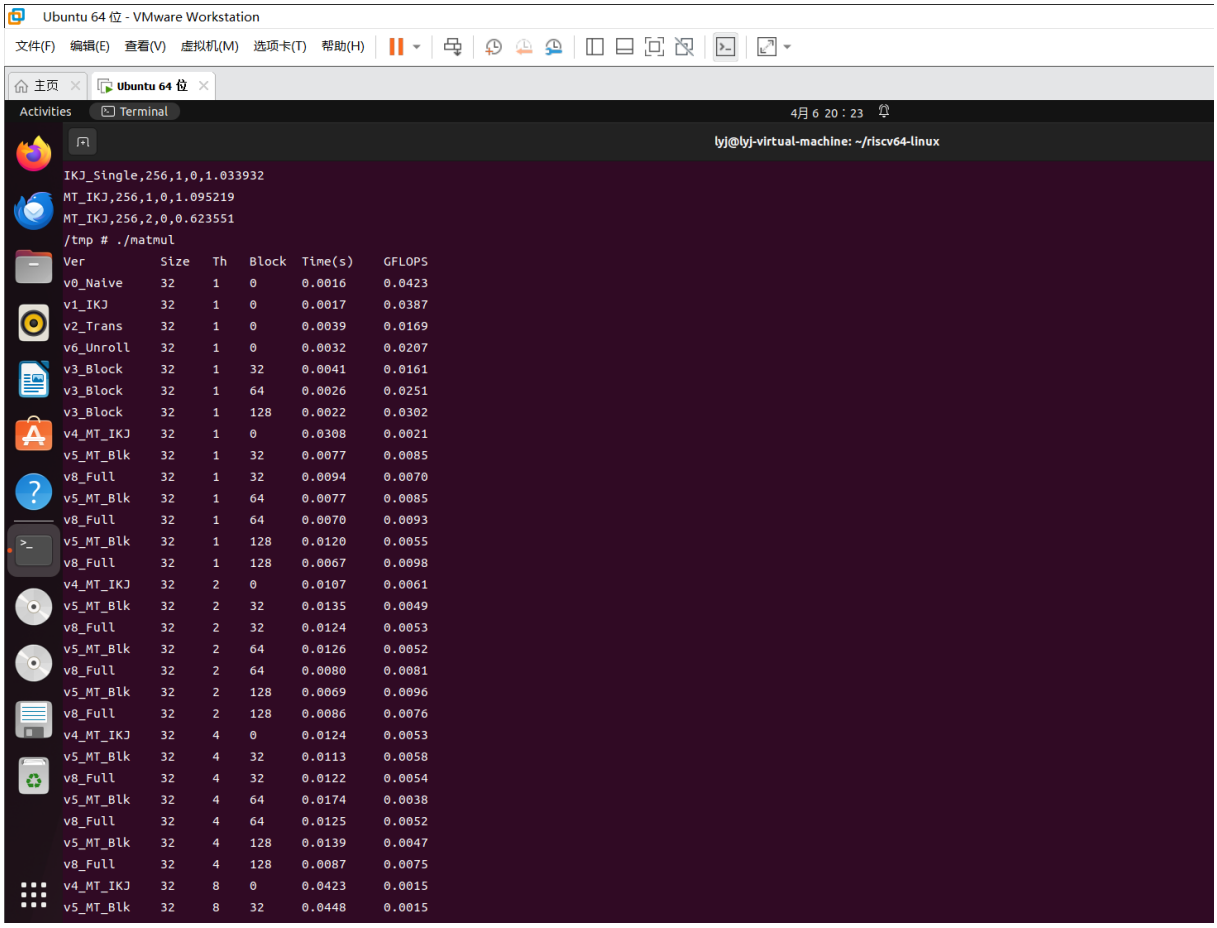


Table 3: 实验一运行过程截图 (VMware)

1.6 实验结果

不同优化版本的矩阵乘法代码在各种测试条件下 (输入规模大小、并行线程数、分块大小) 在 WSL2(Arch Linux) 上运行结果绘制成折线图如图:

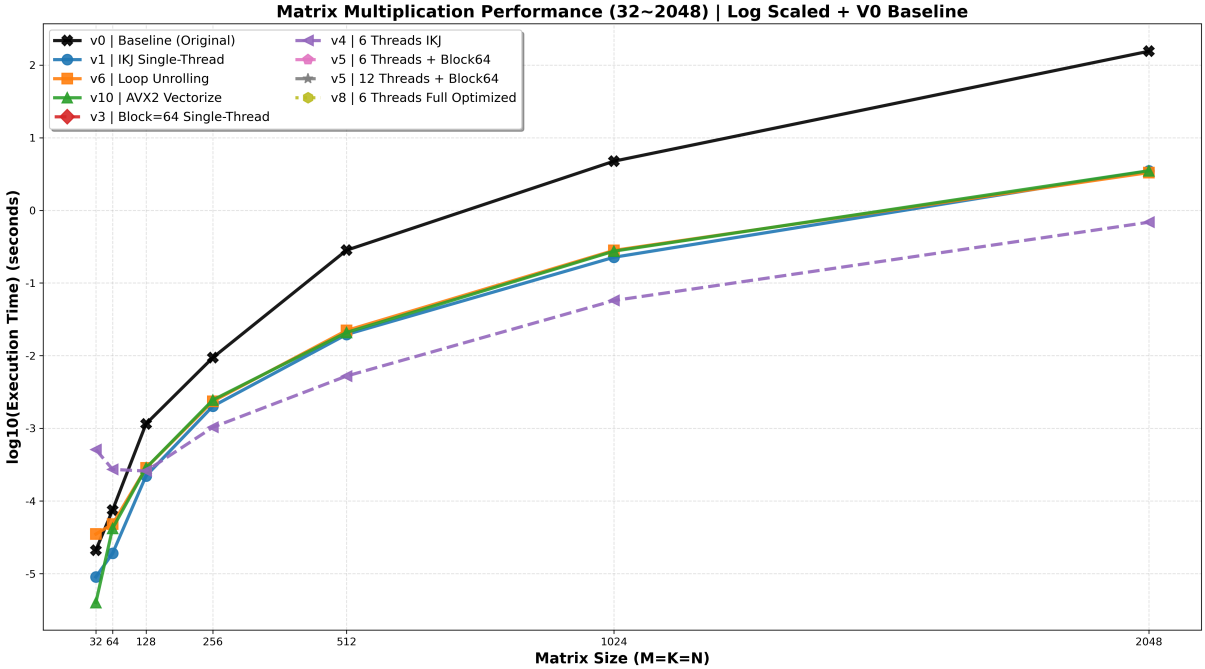


Figure 1: 实验一运行结果折线图 (宿主机)

其中纵坐标为对应运行时间取对数。相应的修改代码在 QEMU 模拟环境上运行结果绘制成折线图如图:

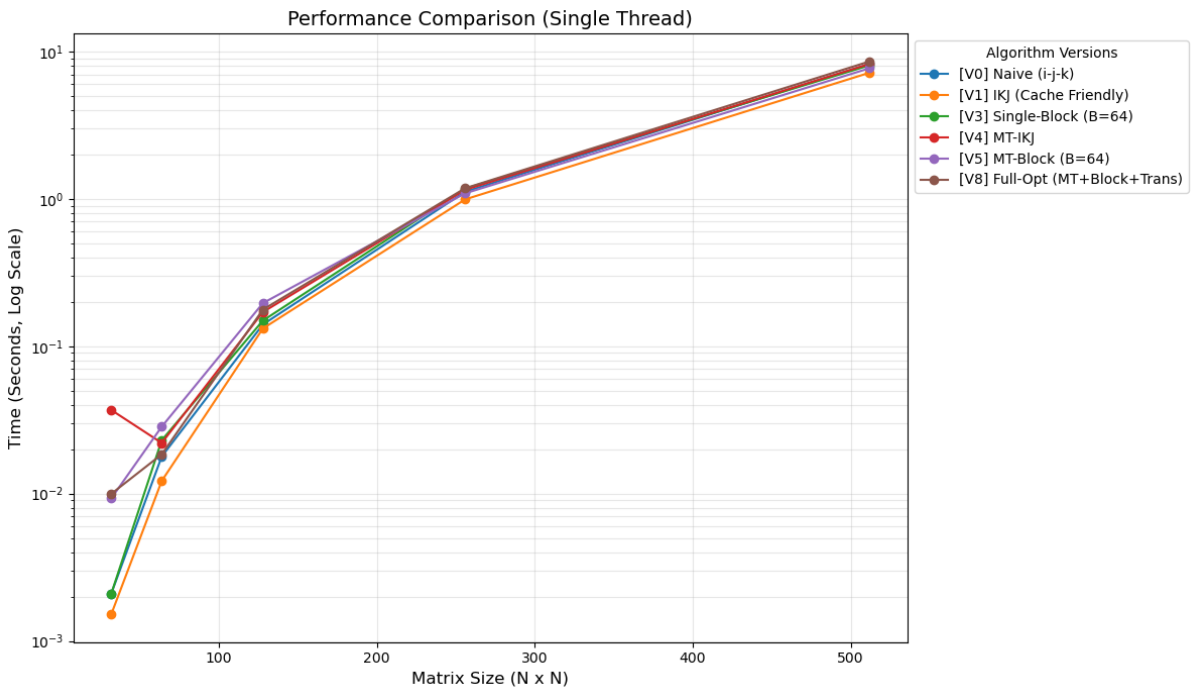


Figure 2: 实验一运行结果折线图 (VMware)

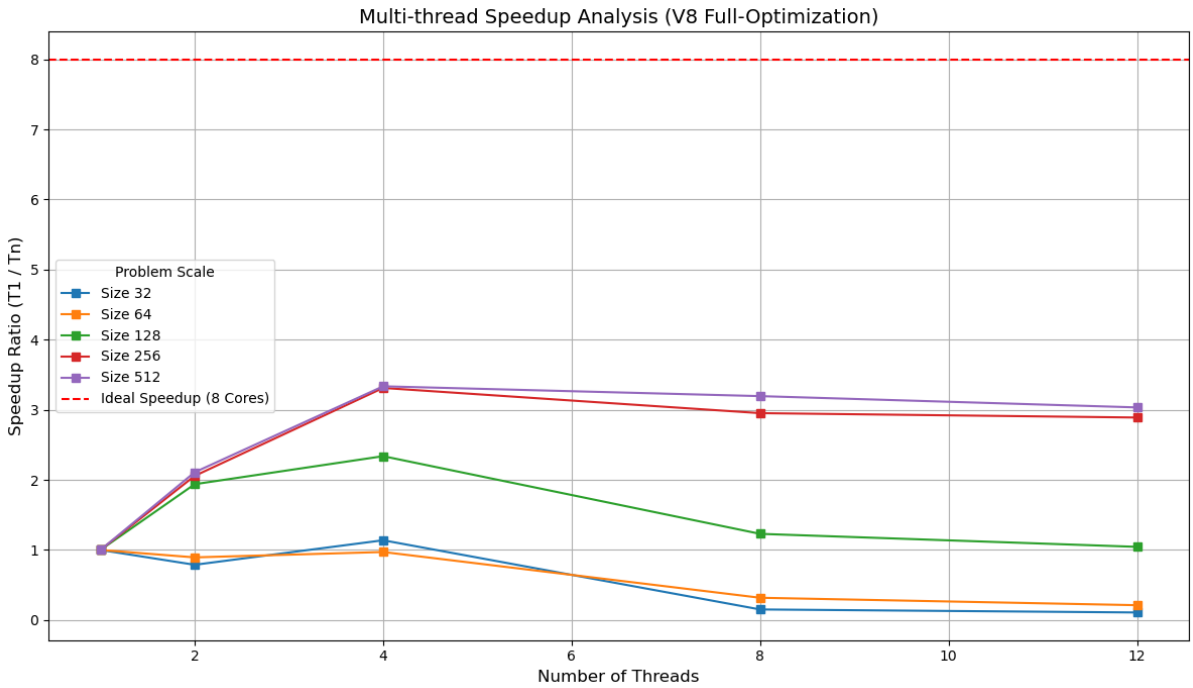


Figure 3: 实验一加速比

从两种环境下的运行结果图片可以看出, 各优化版本的性能较原始版本相比都有了很大的提升

1.7 实验结果分析

- 单线程算法性能分析:
  - 现象: 随着矩阵规模 N 的增大，所有算法的执行时间呈指数级增长。在小规模（32-128）下，各算法差距较小；但在大规模（512-1024）下，Naive (i-j-k) 版本的斜率明显高于其他版本，而 IKJ 和 Loop Unrolling 版本表现出明显的性能优势
  - 原因:Naive 版本在最内层循环访问 B 矩阵时跨度大, 空间局部性差, 导致严重的 Cache Miss。IKJ 通过交换循环次序, 使最内层循环对矩阵 B 的访问变为连续内存空间，极大提高了空间局部性，程序运行效率大大提升。
- 多线并行与加速比分析:
  - 现象:
    * 大计算规模优势: 当规模达到 1024×1024 时, 随着线程数增加, 加速比呈现明显的上升趋势。
    * 小规模" 反向优化": 在 $N = 32$ 或 64 时, 增加线程数不仅没有提速, 加速比反而小于 1(即变慢)。
    * QEMU 瓶颈: 在线程数接近或超过 8 时 (-smp 8), 加速比曲线趋于平缓, 甚至在 $T = 12$ 时略有下滑。

    - WSL 线程与核心数权衡:WSl 中当线程数为 12(大于核心数 6) 时, 运行效率低于线程数为 6 的情况。
  - 原因:
    - 线程开销控制: 小规模任务中, 计算任务本身极快 (微秒级), 而多线程的创建、同步和销毁开销远超计算量, 导致效率下降，此时多线程体现不出优势。
    - 模拟器资源争抢：QEMU 或 WSL2 Arch linux 在宿主机上运行，当模拟线程数超过宿主机物理核心（6 核）或分配核心（8 核）时，会产生剧烈的上下文切换开销，导致系统负载过载。
- 仿真环境与真实环境差异:
  - 现象:WSL 宿主机上的运行时间比 QEMU 低约 2 各数量级;WSL 环境下的曲线更为平稳且符合理论预期;QEMU 下的曲线存在细微波动，且在处理超大规模矩阵时会出现系统调用报错或计时器延迟。
  - 原因:QEMU 作为系统级模拟器，需要将每一条 RISC-V 指令动态翻译为 x86 指令执行，这带来了巨大的软件模拟损耗。同时宿主机拥有真实的硬件缓存和预取器（L1d 288KB），而 QEMU 的缓存层级是软件定义的，无法完美重现硬件级别的访存延迟。
- 其他情况分析:
  - 现象
    - 矩阵转置优化效果并不是很好。
    - 矩阵分块算法优化随着分块的增大, 程序性能可能会降低。
  - 原因
    - 矩阵转置优化版的代码要先执行矩阵优化, 这一步会拖慢矩阵乘法的计算, 即使之后能使得矩阵 B 的访问也是内存连续访问。
    - 当矩阵分块大小超过 L1/L2cache 时, 会使得即使是分块计算矩阵乘法也会出现较大的 Cache Miss 率, 且矩阵分块乘法算法本身较为复杂, 所以实际使用时分块优化的效果也不是最好的。

1.8 实验结论

实验结果证明, 即使是简单调整矩阵乘法内层访问顺序, 使得对所有矩阵都是内存连续式访问, 就能极大增加程序运行效率, 充分体现了空间局部性对程序性能的深刻影响。各种优化方法中循环重排、多线程并行是提升矩阵乘法性能的核心手段, 其中取得最佳优化结果的就是 6 线程并行优化。在多线程应用中，应根据计算规模选择合适的线程数，避免因过载导致的“过饱和”性能损耗, 使得多线程的优势被线程切换等的损耗给掩盖。同时，QEMU 仿真环境能有效验证算法逻辑和相对趋势，但其绝对性能指标受限于软件翻译和宿主机核心资源。

2 实验二

2.1 实验环境

- 硬件环境:
  - 处理器:13th Gen Intel(R) Core(TM) i9-13900H(12CPUs), 2.6GHz
  - 内存:16384MB RAM
- 软件环境:
  - 宿主机操作系统:Windows11
  - VMware Workstation Pro 17:Ubuntu 22.04 LTS 系统,12GB 内存,4 处理器,200GB 硬盘
  - 香山开发环境：xs-env
  - 裸机运行时环境：nexus-am
  - 模拟器：NEMU
  - 编译工具链：RISC-V GNU Compiler Toolchain
  - 目标架构：riscv64-xs

2.2 实验目的

- 基于国产 RISC-V 架构香山仿真环境，掌握 RISC-V GNU Compiler Toolchain、AM 裸机运行时环境和 NEMU 模拟器的基本使用方法，理解程序从编译到运行的完整流程。
- 对附件提供的向量求和程序进行实现与运行，确保程序功能正确，能够得到正确的向量求和结果。
- 通过对程序执行过程和性能指标的观察，掌握程序性能分析的基本方法，理解不同实现方式对程序执行效率的影响。
- 在保证计算结果正确的前提下，对向量求和核心代码进行优化，分析优化前后程序指令流和执行周期的变化。
- 结合循环外提、减少过程调用、循环展开等优化手段，理解基础程序优化方法的原理及其对执行效率的提升作用。
- 通过多种优化方案的实验比较，寻找当前实验环境下性能更优的实现方式，并形成对实验数据和优化结论的分析能力。通过实验对比不同优化策略的效果，加深对程序优化重要性的认识。

2.3 实验原理

任务二的核心对象是向量求和程序，其基本思想是对向量中的各个元素依次进行读取并累加，最终得到整个向量的和。程序正确性的判断依据是：在给定固定输入数据时，优化前后各版本都应输出相同的求和结果。

- 程序性能分析的基本原理: 本实验不仅要求程序能够正确运行，还要求对程序执行效率进行分析。实验中通过观察程序运行过程中的周期数等性能指标，比较不同实现方式下程序的执行开销，从而评价优化方法是否有效。
- 减少过程调用的优化原理: 在优化过程中，可以将原先通过函数调用完成的操作改为直接访问结构体成员和数组元素，例如直接使用 $v->len$ 和 $v->data[i]$。这样可以减少循环中的过程调用次数，缩短执行路径，降低不必要的开销。
- 局部累加变量的优化原理: 若在循环中每次都通过 $*dest = *dest + val$ 更新结果，会引入额外的存储访问。通过设置局部变量 sum 或 t 在寄存器中完成累加，最后再一次性写回目标地址，可以减少循环体中的存储操作，提高运行效率。
- 循环展开的优化原理: 循环展开的思想是每次循环不再只处理一个元素，而是同时处理多个元素。例如 4 路展开或 8 路展开，可以减少循环判断、分支跳转和计数更新等控制开销，使更多指令用于有效计算。
- 多累加器的优化原理: 在循环展开基础上，使用多个独立累加变量，可以减轻单一累加变量带来的数据相关问题，降低顺序依赖对流水线执行效率的限制，从而进一步提升程序性能。这也是优化二和优化四取得更好效果的重要原因。

2.4 实验设计

本实验围绕向量求和程序的性能优化展开，目标是在保证计算结果正确的前提下，比较不同优化方法对程序执行效率的影响。根据实验要求，本任务需要在香山仿真环境中对示例代码进行实现、优化与性能分析，并通过性能指标评价优化效果。实验中统一采用长度为 10000 的整型向量作为输入，使用 `CSR_MCYCLE` 记录核心求和函数的执行周期数，作为主要性能评价指标。

本实验共设计了 6 个版本的向量求和程序，其中包括 1 个基线版本、4 个手工代码优化版本和 1 个编译器优化尝试版本。各版本的设计思路如表所示：

Table 4: 向量求和优化版本迭代说明

版本分类	核心技术	优化逻辑说明
基线版本	原始逐元素求和	使用 `vec_length()` 与 `get_vec_element()` 完成循环
优化一	直接数组访问 + 局部累加	去除不必要的过程调用，直接访问 `v->data`，并使用
优化二	4 路循环展开	每轮循环同时处理 4 个元素，并设置 4 个独立累加器，
优化三	指针遍历 + 4 路展开	在优化二基础上改用指针遍历，希望减少地址计算开
优化四	8 路累加器 + 8 路展开	进一步扩大循环展开规模，利用更多独立累加器增强
编译器优化尝试	函数级 O3	在优化四基础上增加 `__attribute__((optimize("O3`

2.5 实验过程

首先，按照实验文档附录 3 完成香山仿真环境的配置与验证工作。具体包括克隆 `xs-env` 仓库，执行 `setup-tools.sh` 安装依赖，使用 `source setup.sh` 和 `source env.sh` 配置环境变量，并检查相关路径是否正确加载。之后进入 `nexus-am/apps/hello` 目录，将实验代码放入 `hello.c` 中，通过 `make ARCH=riscv64-xs` 进行编译，生成对应的 `.elf`、`.bin` 和反汇编文件；再进入 `NEMU` 目录，使用 `riscv64-nemu-interpreter` 加载生成的二进制镜像进行运行。通过这一过程，验证了任务二代码能够在香山仿真环境中被正确编译和执行，为后续优化实验奠定了基础。

在确认程序能够正常运行后，首先测试基线版本。基线版本采用最原始的向量求和实现方式，在循环中通过 `vec_length(v)` 获取向量长度，并通过 `get_vec_element()` 获取每一个元素，然后再逐项累加到结果变量中。该写法逻辑清晰，便于验证程序正确性，但存在明显的函数调用、边界检查和频繁存储访问开销，因此将其作为后续所有优化版本的参照标准。实验结果表明，基线版本能够正确输出向量和 50005000，并测得核心求和函数的周期数为 50012。

下面给出基线版本运行结果截图，用于说明原始版本程序能够正确执行，并为后续优化提供初始性能基准。

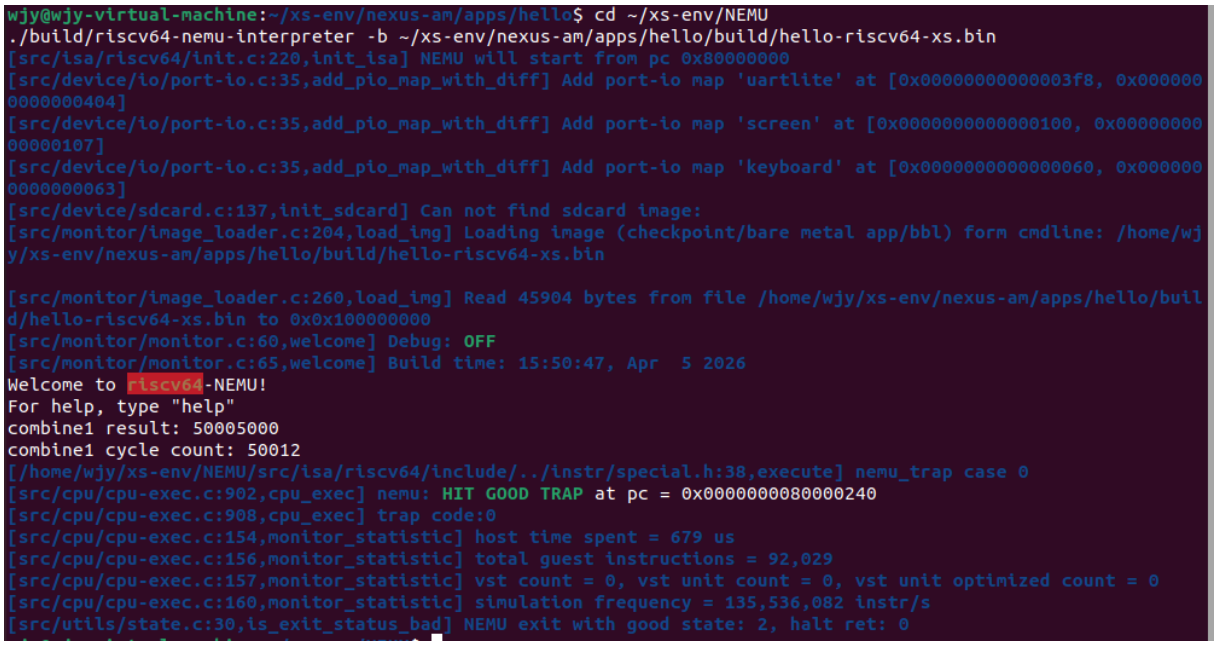


Figure 4: 任务二原始版本运行结果截图

在得到基线结果后，开始依次进行多轮手工代码优化。

第一轮优化的核心思想是减少不必要的过程调用，直接使用 `v->len` 和 `v->data` 访问向量长度和数组内容，并使用局部变量完成累加后再统一写回结果。这样可以有效减少循环体中的函数调用与中间访存开销。

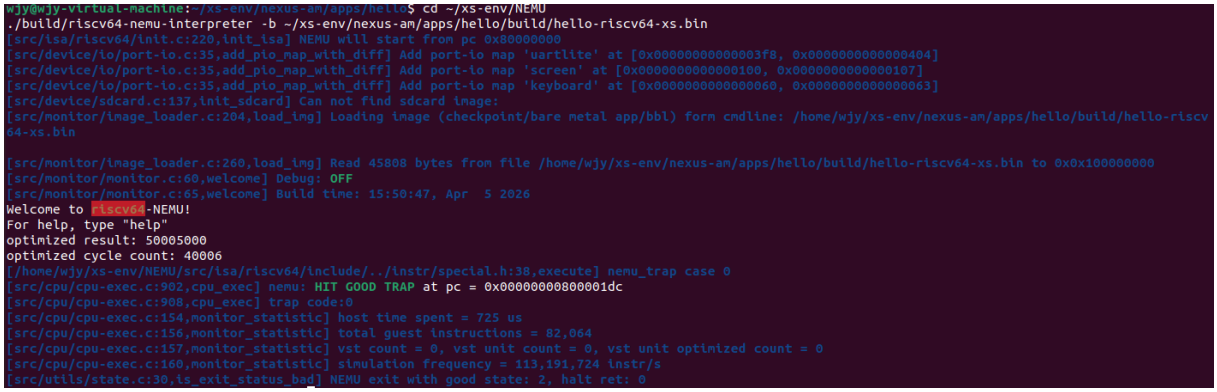


Figure 5: 任务二优化一运行结果截图

第二轮优化在此基础上引入 4 路循环展开，使每轮循环同时处理 4 个元素，并使用多个独立累加器减轻顺序相关带来的性能限制。实验结果显示，优化二的周期数明显下降，说明循环展开在当前实验环境下具有显著效果。

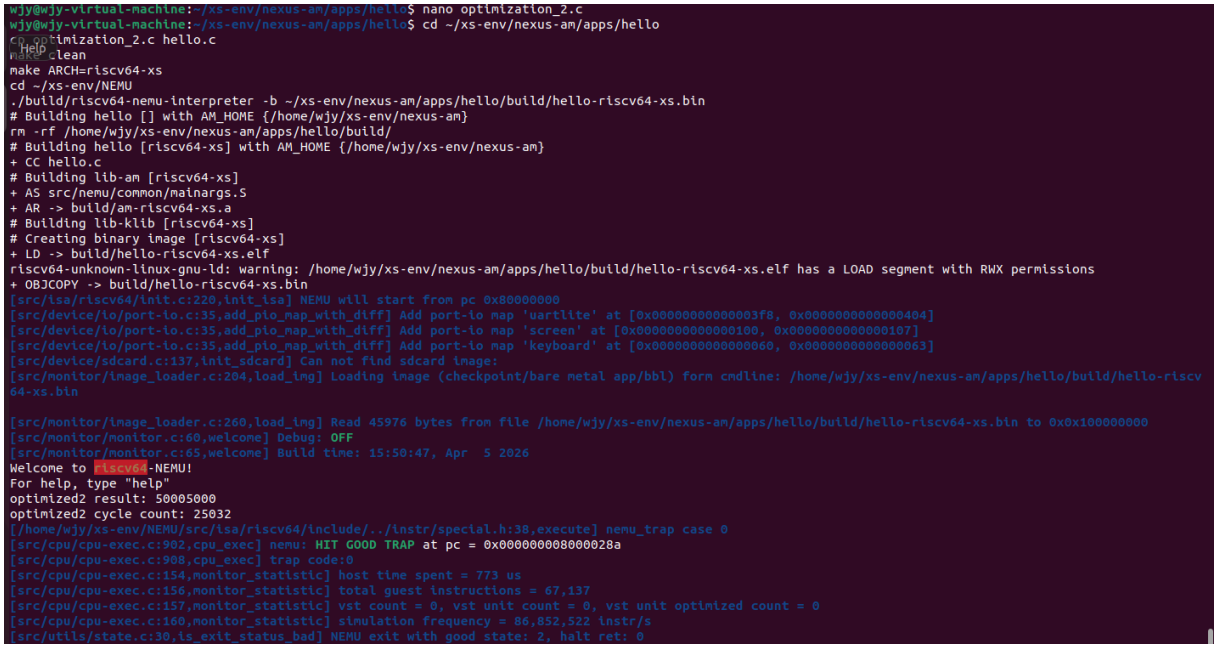
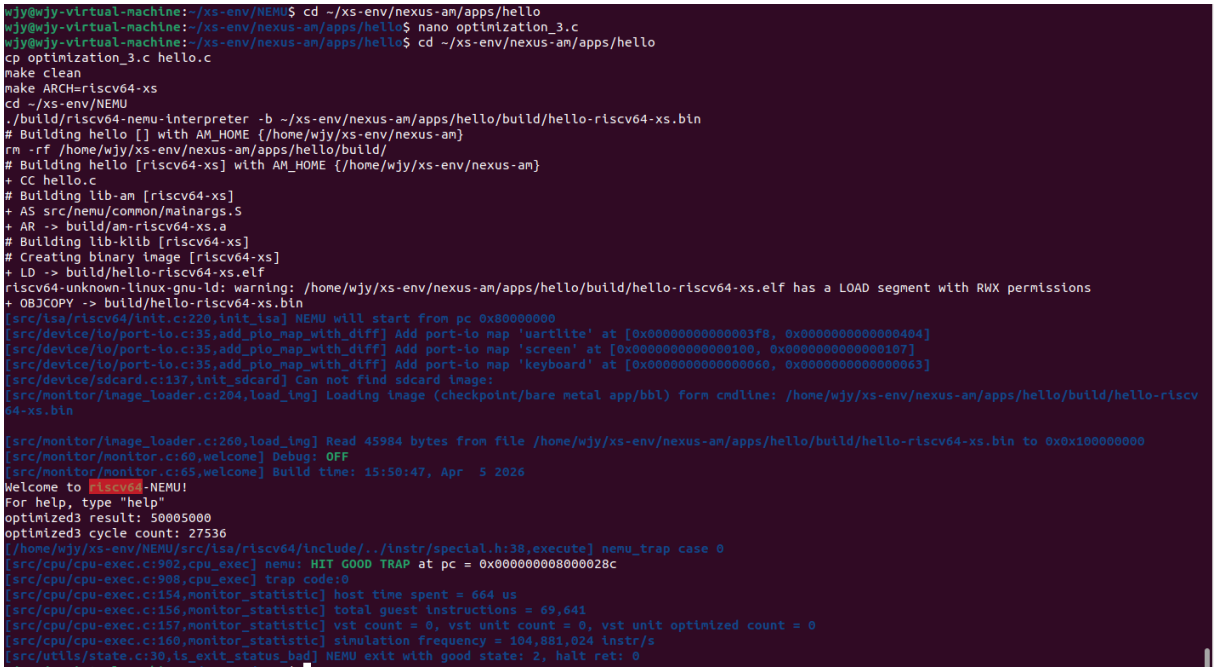


Figure 6: 任务二优化二运行结果截图

在优化二之后，又继续尝试优化三和优化四。

优化三在优化二基础上改用指针遍历并保留 4 路展开，希望进一步减少地址计算开销，但实际结果表明其周期数反而略高于优化二，说明这种改写在当前环境下并未优于数组下标访问。

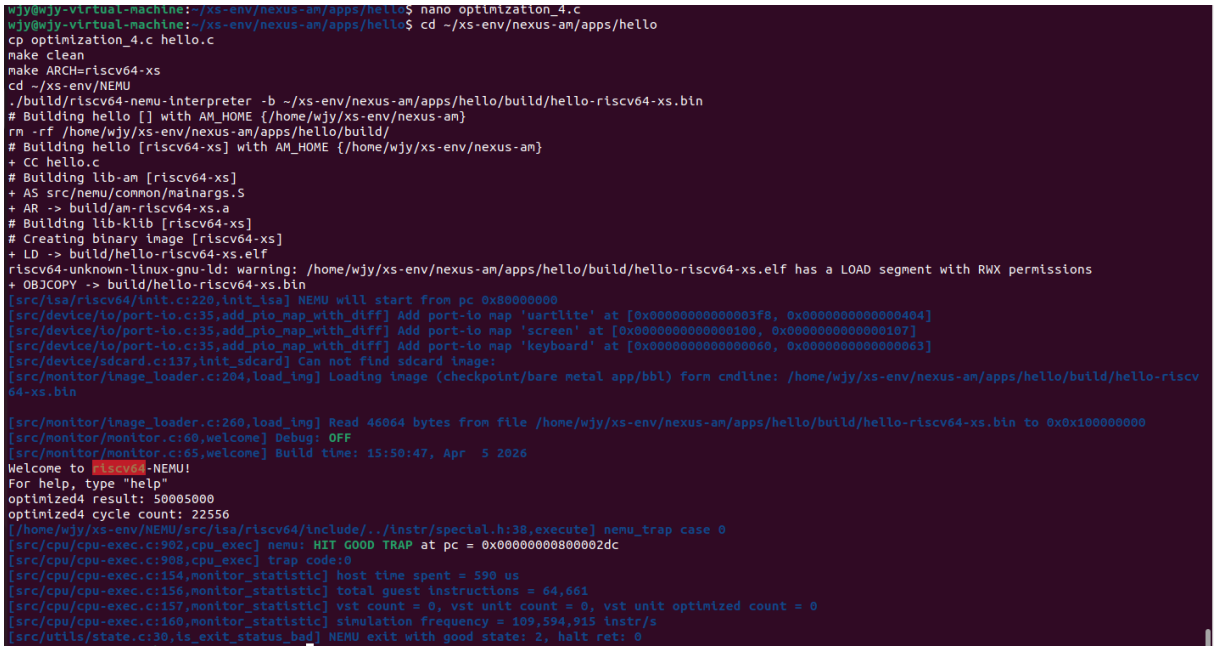


```
wjy@wjy-virtual-machine:~/xs-env/NEMU$ cd ~/xs-env/nexus-am/apps/hello
wjy@wjy-virtual-machine:~/xs-env/nexus-am/apps/hello$ nano optimization_3.c
wjy@wjy-virtual-machine:~/xs-env/nexus-am/apps/hello$ cd ~/xs-env/nexus-am/apps/hello
cp optimization_3.c hello.c
make clean
make ARCH=riscv64-xs
cd ~/xs-env/NEMU
./build/riscv64-nemu-interpreter -b ~/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.bin
# Building hello [] with AM_HOME {/home/wjy/xs-env/nexus-am}
rm -rf /home/wjy/xs-env/nexus-am/apps/hello/build/
# Building hello [riscv64-xs] with AM_HOME {/home/wjy/xs-env/nexus-am}
+ CC hello.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-riscv64-xs.elf
riscv64-unknown-linux-gnu-ld: warning: /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.elf has a LOAD segment with RWX permissions
+ OBJCOPY -> build/hello-riscv64-xs.bin
[src/isa/riscv64/init.c:220,init_isa] NEMU will start from pc 0x80000000
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'uartlite' at [0x00000000000003f8, 0x0000000000000404]
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'screen' at [0x0000000000000100, 0x0000000000000107]
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'keyboard' at [0x0000000000000060, 0x0000000000000063]
[src/device/sdcard.c:137,init_sdcard] Can not find sdcard image:
[src/monitor/image_loader.c:204,load_img] Loading image (checkpoint/bare metal app/bbl) form cmdline: /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv
64-xs.bin

[src/monitor/image_loader.c:260,load_img] Read 45984 bytes from file /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.bin to 0x0x100000000
[src/monitor/monitor.c:60,welcome] Debug: OFF
[src/monitor/monitor.c:65,welcome] Build time: 15:50:47, Apr  5 2026
Welcome to riscv64-NEMU!
For help, type "help"
optimized3 result: 50005000
optimized3 cycle count: 27536
[/home/wjy/xs-env/NEMU/src/isa/riscv64/include/../instr/special.h:38,execute] nemu_trap case 0
[src/cpu/cpu-exec.c:902,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008000028c
[src/cpu/cpu-exec.c:908,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:154,monitor_statistic] host time spent = 664 us
[src/cpu/cpu-exec.c:156,monitor_statistic] total guest instructions = 69,641
[src/cpu/cpu-exec.c:157,monitor_statistic] vst count = 0, vst unit count = 0, vst unit optimized count = 0
[src/cpu/cpu-exec.c:160,monitor_statistic] simulation frequency = 104,881,024 instr/s
[src/utils/state.c:30,is_exit_status_bad] NEMU exit with good state: 2, halt ret: 0
```

Figure 7: 任务二优化三运行结果截图

随后进一步实现优化四，在优化二思路上扩展为 8 路累加器与 8 路循环展开。该版本继续降低了循环控制开销和单一累加变量带来的顺序相关，使周期数进一步下降，最终成为本实验中性能最好的实现。下面给出优化四运行结果截图，用于说明当前最佳版本在保持结果正确的前提下，已经获得了最低的周期数。



```
wjy@wjy-virtual-machine:~/xs-env/nexus-am/apps/hello$ nano optimization_4.c
wjy@wjy-virtual-machine:~/xs-env/nexus-am/apps/hello$ cd ~/xs-env/nexus-am/apps/hello
cp optimization_4.c hello.c
make clean
make ARCH=riscv64-xs
cd ~/xs-env/NEMU
./build/riscv64-nemu-interpreter -b ~/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.bin
# Building hello [] with AM_HOME {/home/wjy/xs-env/nexus-am}
rm -rf /home/wjy/xs-env/nexus-am/apps/hello/build/
# Building hello [riscv64-xs] with AM_HOME {/home/wjy/xs-env/nexus-am}
+ CC hello.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-riscv64-xs.elf
riscv64-unknown-linux-gnu-ld: warning: /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.elf has a LOAD segment with RWX permissions
+ OBJCOPY -> build/hello-riscv64-xs.bin
[src/isa/riscv64/init.c:220,init_isa] NEMU will start from pc 0x80000000
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'uartlite' at [0x00000000000003f8, 0x0000000000000404]
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'screen' at [0x0000000000000100, 0x0000000000000107]
[src/device/io/port-io.c:35,add_pio_map_with_diff] Add port-io map 'keyboard' at [0x0000000000000060, 0x0000000000000063]
[src/device/sdcard.c:137,init_sdcard] Can not find sdcard image:
[src/monitor/image_loader.c:204,load_img] Loading image (checkpoint/bare metal app/bbl) form cmdline: /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv
64-xs.bin

[src/monitor/image_loader.c:260,load_img] Read 46064 bytes from file /home/wjy/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.bin to 0x0x100000000
[src/monitor/monitor.c:60,welcome] Debug: OFF
[src/monitor/monitor.c:65,welcome] Build time: 15:50:47, Apr  5 2026
Welcome to riscv64-NEMU!
For help, type "help"
optimized4 result: 50005000
optimized4 cycle count: 22556
[/home/wjy/xs-env/NEMU/src/isa/riscv64/include/../instr/special.h:38,execute] nemu_trap case 0
[src/cpu/cpu-exec.c:902,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x00000000800002dc
[src/cpu/cpu-exec.c:908,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:154,monitor_statistic] host time spent = 590 us
[src/cpu/cpu-exec.c:156,monitor_statistic] total guest instructions = 64,661
[src/cpu/cpu-exec.c:157,monitor_statistic] vst count = 0, vst unit count = 0, vst unit optimized count = 0
[src/cpu/cpu-exec.c:160,monitor_statistic] simulation frequency = 109,594,915 instr/s
[src/utils/state.c:30,is_exit_status_bad] NEMU exit with good state: 2, halt ret: 0
```

Figure 8: 任务二优化四运行结果截图

在完成四轮手工优化之后，为了进一步考察编译器优化的作用，又在不改变优化四算法结构的前提下，对核心函数增加了 `__attribute__((optimize("O3")))` 属性，尝

试通过更高等级的编译器优化进一步压缩指令流并提升性能。实验结果表明，加入函数级 O3 后，程序输出结果仍然正确，但周期数与优化四相比未发生改变。这说明当前优化四代码本身已经较适合编译器自动优化，其结构已较接近当前实验环境下的高效实现形式，因此编译器优化在这一阶段未带来额外显著收益。

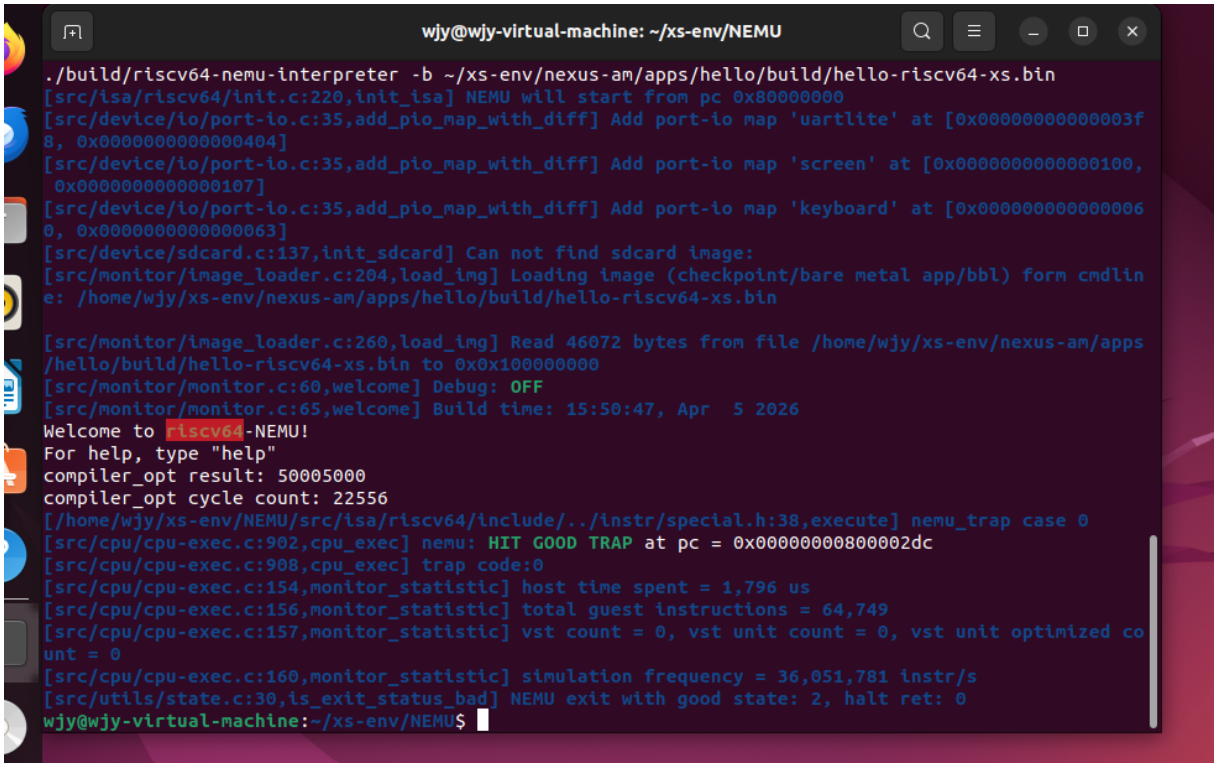


Figure 9: 任务二编译器优化尝试结果截图

综合整个实验过程可以看出，本任务遵循了“先验证正确性，再逐步优化，再补充编译器优化分析”的推进顺序。基线版本用于建立参照标准；优化一至优化四用于考察减少过程调用、消除不必要存储引用、循环展开和多累加器设计等手工优化手段对性能的影响；编译器优化尝试则用于补充分析当前代码在更高优化等级下的表现。通过这样的实验过程设计，可以较清楚地观察不同优化方法对向量求和程序执行效率的影响。

2.6 实验结果

各版本程序在相同输入规模下运行后，均得到了正确的向量求和结果 50005000，说明优化过程没有破坏程序正确性。不同优化版本的周期数统计结果如表所示：

Table 5: 向量求和不同优化版本性能对比

版本	优化方法	运行结果	周期数
基线版本	原始核心求和实现	50005000	50012
优化一	直接数组访问 + 局部累加	50005000	40006
优化二	4 路循环展开	50005000	25032
优化三	指针遍历 + 4 路展开	50005000	27536
优化四	8 路累加器 + 8 路展开	50005000	22556
编译器优化尝试	优化四基础上增加函数级 O3	50005000	22556

2.7 实验结果分析

- 基线版本与优化一分析:
  - 现象: 基线版本的周期数为 `50012`，优化一的周期数下降到 `40006`，在保证结果仍然为 `50005000` 的前提下，性能得到了明显提升。
  - 原因: 基线版本在循环中需要频繁调用 `vec_length(v)` 和 `get_vec_element()`，同时还存在较多的边界检查和不必要的中间访存开销。优化一通过直接访问 `v->len` 和 `v->data`，并使用局部变量完成累加后统一写回，减少了过程调用和存储访问次数，因此执行效率明显提高。
- 优化二分析:
  - 现象: 在优化一基础上引入 4 路循环展开后，周期数进一步下降到 `25032`，相比优化一有了更加显著的提升，说明该优化在当前实验环境下效果十分明显。
  - 原因:4 路循环展开使每轮循环能够同时处理 4 个元素，从而减少了循环控制指令（如循环判断、跳转、计数更新）所占比例。同时设置多个独立累加器后，可以减轻单一累加变量带来的顺序相关问题，使程序更容易发挥指令级并行能力，因此整体性能得到了大幅改善。
- 优化三分析:
  - 现象: 优化三在优化二基础上改用指针遍历并保留 4 路展开后，周期数变为 `27536`，反而略高于优化二的 `25032`，说明该优化没有继续带来性能提升。
  - 原因: 虽然指针遍历在理论上可能减少部分地址计算开销，但在当前 RISC-V 裸机仿真环境与编译条件下，这种写法并未优于数组下标访问。相反，优化三引入的额外控制逻辑可能抵消了部分收益，因此其实际运行效果不如优化二。这也说明并不是所有“更底层”的写法都一定更快，优化效果仍需通过实验数据验证。
- 优化四分析:
  - 现象: 优化四在优化二思路基础上进一步扩展为 8 路累加器和 8 路循环展开，周期数继续下降到 `22556`，成为所有测试版本中的最佳结果。
  - 原因: 优化四相比优化二进一步扩大了循环展开规模，并增加了独立累加器数量，使循环控制开销进一步减少，同时也进一步降低了顺序相关对流水线执行效率的限制。因此，该版本在当前实验环境下能够获得最好的性能表现。

- 编译器优化尝试分析:
  - 现象: 在优化四基础上增加函数级 O3 编译器优化属性后，程序输出结果仍然正确，但周期数与优化四相比没有发生变化，说明未观察到额外明显收益。
  - 原因: 这说明优化四本身已经较充分地利用了当前编译器可发挥的优化空间，其代码结构已经较接近当前实验环境下的高效实现形式。因此，继续显式增加 O3 并没有带来进一步的可观提升。本实验中的主要性能收益仍然来自手工代码优化，而不是额外加入的编译器优化属性。
- 总体结果分析:
  - 现象: 从基线版本到优化四，周期数由 50012 逐步下降到 22556，说明总体优化过程是有效的；其中优化一、优化二和优化四均取得了明显提升，而优化三和编译器优化尝试未带来进一步改善。
  - 原因: 本实验中性能提升的主要来源包括减少过程调用、降低不必要的存储引用、采用循环展开以及增加独立累加器数量等手工代码优化方法。其中，优化四在减少循环控制开销和提高指令级并行性方面效果最好，因此成为最终性能最优的实现版本。
- 最终结论:
  - 若以基线版本作为参照，则优化四相对基线的性能提升约为

$$\frac{50012 - 22556}{50012} \approx 54.9\%$$

  - 这表明，在保证程序计算结果正确的前提下，优化四将核心求和过程的执行周期降低了约 **54.9%**，是本实验中最有效的优化方案。

2.8 实验结论

实验结果表明, 即使只是对向量求和程序进行较为基础的代码优化, 如减少过程调用、直接访问数组元素、使用局部变量累加等, 也能够显著提升程序运行效率, 充分体现了程序组织方式和访存方式对性能的直接影响。

在多种优化方法中, 循环展开和多累加器设计是提升向量求和性能的核心手段。其中取得最佳优化效果的是优化四, 即 8 路累加器与 8 路循环展开版本。该版本在保证计算结果正确的前提下, 将核心求和过程的周期数由基线版本的 50012 降低到 22556, 性能提升约为 **54.9%**。

同时, 实验也表明并非所有“更底层”的写法都一定更优。例如, 优化三虽然尝试将数组下标访问改为指针遍历, 但实际性能反而略低于优化二。这说明程序优化必须结合具体平台、编译器和运行环境, 通过实验数据进行验证, 而不能仅凭理论推测判断。

此外, 在优化四基础上增加函数级 O3 编译器优化属性后, 程序性能未出现进一步明显提升。这说明当前优化四代码本身已经较适合编译器自动优化, 其结构已较接近当前实验环境下的高效实现形式。因此, 本实验中的主要性能收益仍然来自手工代码优化, 而编译器优化在该阶段未表现出额外显著作用。

总体来看, 香山仿真环境和 NEMU 模拟器能够有效支持任务二中向量求和程序的实现、优化与性能分析。虽然仿真环境下的绝对性能会受到软件模拟开销影响, 但其对于验证优化思路、观察性能变化趋势以及分析不同实现方式之间的相对效果仍然具有重要意义。

A 实验相关代码

本附录给出实验中使用的关键 C 语言代码文件，便于展示程序实现与优化过程。

A.1 任务二基线版本代码

Listing 1: 任务二基线版本代码

```c
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

int get_vec_element(vec_ptr v, long index, data_t *dest) {
    if (index < 0 || index >= v->len)
        return 0;
    *dest = v->data[index];
    return 1;
}

long vec_length(vec_ptr v) {
    return v->len;
}

/* 原始版本 */
void combine1(vec_ptr v, data_t *dest) {
    long i;
```

```
    *dest = 0;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest + val;
    }
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine1(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;

    printf("combine1 result: %d\n", result);
    printf("combine1 cycle count: %lu\n", cycles);

    return 0;
}
```

A.2 任务二优化一代码

Listing 2: 任务二优化一代码

```
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

```

```c
#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

/* 优化后的向量求和 */
void combine_optimized(vec_ptr v, data_t *dest) {
    long i;
    long length = v->len;       // 长度外提
    data_t *data = v->data;     // 数据指针外提
    data_t sum = 0;             // 局部累加变量

    for (i = 0; i < length; i++) {
        sum += data[i];
    }

    *dest = sum;
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

```

```
    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine_optimized(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;

    printf("optimized result: %d\n", result);
    printf("optimized cycle count: %lu\n", cycles);

    return 0;
}
```

A.3 任务二优化二代码

Listing 3: 任务二优化二代码

```
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

/* 优化二: 直接数组访问 + 局部累加 + 4 路循环展开 */
void combine_optimized2(vec_ptr v, data_t *dest) {
    long length = v->len;
```

```
    data_t *data = v->data;

    long i = 0;
    long limit = length - (length % 4);

    data_t sum0 = 0;
    data_t sum1 = 0;
    data_t sum2 = 0;
    data_t sum3 = 0;

    for (; i < limit; i += 4) {
        sum0 += data[i];
        sum1 += data[i + 1];
        sum2 += data[i + 2];
        sum3 += data[i + 3];
    }

    data_t sum = sum0 + sum1 + sum2 + sum3;

    for (; i < length; i++) {
        sum += data[i];
    }

    *dest = sum;
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine_optimized2(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;
```

```

    printf("optimized2 result: %d\n", result);
    printf("optimized2 cycle count: %lu\n", cycles);

    return 0;
}
```

A.4 任务二优化三代码

Listing 4: 任务二优化三代码

```
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

/* 优化三: 局部累加 + 4 路展开 + 指针遍历 */
void combine_optimized3(vec_ptr v, data_t *dest) {
    long length = v->len;
    data_t *ptr = v->data;

    data_t sum0 = 0;
    data_t sum1 = 0;
    data_t sum2 = 0;
    data_t sum3 = 0;

    long cnt = length / 4;
```

```c
    while (cnt--) {
        sum0 += ptr[0];
        sum1 += ptr[1];
        sum2 += ptr[2];
        sum3 += ptr[3];
        ptr += 4;
    }

    data_t sum = sum0 + sum1 + sum2 + sum3;

    long remain = length % 4;
    while (remain--) {
        sum += *ptr;
        ptr++;
    }

    *dest = sum;
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine_optimized3(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;

    printf("optimized3 result: %d\n", result);
    printf("optimized3 cycle count: %lu\n", cycles);

    return 0;
}
```

A.5 任务二优化四代码

Listing 5: 任务二优化四代码

```
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

/* 优化四：提高并行性，使用 8 路累加器 */
void combine_optimized4(vec_ptr v, data_t *dest) {
    long length = v->len;
    data_t *data = v->data;

    long i = 0;
    long limit = length - (length % 8);

    data_t sum0 = 0;
    data_t sum1 = 0;
    data_t sum2 = 0;
    data_t sum3 = 0;
    data_t sum4 = 0;
    data_t sum5 = 0;
    data_t sum6 = 0;
    data_t sum7 = 0;

    for (; i < limit; i += 8) {
        sum0 += data[i];
```

```
        sum1 += data[i + 1];
        sum2 += data[i + 2];
        sum3 += data[i + 3];
        sum4 += data[i + 4];
        sum5 += data[i + 5];
        sum6 += data[i + 6];
        sum7 += data[i + 7];
    }

    data_t sum = sum0 + sum1 + sum2 + sum3 +
                 sum4 + sum5 + sum6 + sum7;

    for (; i < length; i++) {
        sum += data[i];
    }

    *dest = sum;
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine_optimized4(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;

    printf("optimized4 result: %d\n", result);
    printf("optimized4 cycle count: %lu\n", cycles);

    return 0;
}
```

A.6 任务二编译器优化尝试代码

Listing 6: 任务二编译器优化尝试代码

```
#include <am.h>
#include <klib.h>
#include <klib-macros.h>
#include <csr.h>

typedef int data_t;

#define MAX_VEC_LEN 10000
static data_t static_data[MAX_VEC_LEN];

typedef struct {
    long len;
    data_t *data;
} vec_rec, *vec_ptr;

vec_ptr new_vec(long len) {
    static vec_rec static_vec;

    if (len <= 0 || len > MAX_VEC_LEN) {
        return NULL;
    }

    static_vec.len = len;
    static_vec.data = static_data;
    return &static_vec;
}

/* 编译器优化版本：在优化四基础上，强制该函数按 O3 编译 */
__attribute__((optimize("O3")))
void combine_compiler_opt(vec_ptr v, data_t *dest) {
    long length = v->len;
    data_t *data = v->data;

    long i = 0;
    long limit = length - (length % 8);

    data_t sum0 = 0;
    data_t sum1 = 0;
    data_t sum2 = 0;
    data_t sum3 = 0;
    data_t sum4 = 0;
    data_t sum5 = 0;
    data_t sum6 = 0;
    data_t sum7 = 0;

    for (; i < limit; i += 8) {
```

```
        sum0 += data[i];
        sum1 += data[i + 1];
        sum2 += data[i + 2];
        sum3 += data[i + 3];
        sum4 += data[i + 4];
        sum5 += data[i + 5];
        sum6 += data[i + 6];
        sum7 += data[i + 7];
    }

    data_t sum = sum0 + sum1 + sum2 + sum3 +
                 sum4 + sum5 + sum6 + sum7;

    for (; i < length; i++) {
        sum += data[i];
    }

    *dest = sum;
}

int main() {
    _ioe_init();

    long len = 10000;
    vec_ptr v = new_vec(len);

    if (!v) {
        printf("Failed to create vector\n");
        return 1;
    }

    for (long i = 0; i < len; i++) {
        v->data[i] = i + 1;
    }

    data_t result;
    uint64_t start_cycle, end_cycle, cycles;

    start_cycle = csr_read(CSR_MCYCLE);
    combine_compiler_opt(v, &result);
    end_cycle = csr_read(CSR_MCYCLE);

    cycles = end_cycle - start_cycle;

    printf("compiler_opt result: %d\n", result);
    printf("compiler_opt cycle count: %lu\n", cycles);

    return 0;
}
```